

Microservices Architecture

Deep dive into each service — design, API, and data model

1. Microservices Design Principles

Each K Star service follows the Single Responsibility Principle — one service, one business domain. Services do not share databases. They communicate via HTTP (synchronous, for query responses) through the API Gateway, and via Kafka topics (asynchronous, for state-changing events).

Principle	How Applied
Single Responsibility	One domain per service (auth, exchange, transfer, nft, nacha)
Database per Service	MySQL for relational, MongoDB for document — no shared schemas
API Gateway	All client traffic enters via port 8080 only
Async Messaging	State changes publish Kafka events; consumers react independently
Stateless Services	JWT tokens carry identity; no server-side session state
Containerization	Each service has its own Dockerfile; compose orchestrates all

2. Gateway Service (Port 8080)

The Gateway Service is the only component exposed to the internet. Built on Spring Cloud Gateway (WebFlux reactive), it validates JWT tokens and routes requests to the correct downstream service.

■ AuthFilter — How JWT Validation Works

- Request arrives at gateway on any path
- AuthFilter checks if path is in PUBLIC_PATHS (/api/auth/login, /api/auth/register)
- Public paths: pass through immediately without token check
- Protected paths: extract "Authorization: Bearer " header
- JwtUtil.isTokenValid() parses and verifies HMAC-SHA256 signature
- If valid: extract username, add X-User-Name header to request, forward downstream
- If invalid: return 401 Unauthorized immediately

■ Route Configuration

```
spring.cloud.gateway.routes:
```

```
- id: auth-service
```

```

uri: http://user-service:8081
predicates: [Path=/api/auth/**] # Public – no AuthFilter
- id: user-service
uri: http://user-service:8081
predicates: [Path=/api/users/**] # Protected – AuthFilter applied
filters: [AuthFilter]
- id: exchange-service
uri: http://exchange-service:8082
predicates: [Path=/api/exchange/**]
filters: [AuthFilter]
- id: nacha-service
uri: http://nacha-service:8085
predicates: [Path=/api/nacha/**]
filters: [AuthFilter]

```

3. User Service (Port 8081)

Manages user registration, authentication, and multi-currency wallets. It is the identity provider for the entire system. On registration, three default wallets (INR ₹10,000, USD \$0, EUR €0) are created automatically.

■ Key Endpoints

Method	Path	Auth	Description
POST	/api/auth/register	Public	Register new user, create wallets, return JWT
POST	/api/auth/login	Public	Authenticate user, return JWT token
GET	/api/users/me	JWT	Get authenticated user profile + wallets
GET	/api/users/{id}/wallets	JWT	List all wallets for user
POST	/api/users/{id}/wallets?currency=EUR	JWT	Create a new currency wallet
GET	/api/users/me/balance	JWT	Get total portfolio in INR equivalent

■ Database Tables

Table	Key Columns	Notes
users	id, username, email, password, phone, role, created_at	password is not hashed password
wallets	id, user_id, currency, balance, upi_id, created_at	One row per currency per user

4. Exchange Service (Port 8082)

Handles real-time currency conversion. Exchange rates are stored in MongoDB and updated on a scheduled interval. Conversion history is recorded per user. The service also publishes rate change events to Kafka so other services can react to rate updates.

■ Key Endpoints

Method	Path	Description
POST	/api/exchange/convert	Convert currency (from, to, amount)
GET	/api/exchange/rates	Get all current live rates
GET	/api/exchange/rates/{currency}	Get rate for specific currency
GET	/api/exchange/history	Get user conversion history

■ Supported Currency Pairs

From	To	Approx Rate
USD	INR	84.12
USD	EUR	0.92
USD	GBP	0.79
EUR	INR	91.6
EUR	USD	1.087
GBP	INR	106.7
INR	USD	0.0119
JPY	INR	0.556

5. Transfer Service (Port 8083)

Handles UPI-based P2P money transfers. Each transfer publishes a Kafka event, which the same service consumes to process the actual debit/credit. This event-driven design ensures the payment is idempotent and auditable.

■ Kafka Event Flow

- User submits transfer (fromUpId, toUpId, amount, upiPin)
- TransferService validates UPI IDs and PIN
- TransferService publishes TransferEvent to Kafka topic "transfer-events"
- NotificationConsumer consumes the event and logs notification
- Transaction record saved to MySQL with status COMPLETED or FAILED
- Response returned to user with transaction ID

■ Key Endpoints

Method	Path	Description
POST	/api/transfer/upi	Initiate UPI P2P transfer
GET	/api/transfer/history?upId={id}	Get transaction history for UPI ID
GET	/api/transfer/{txnId}	Get specific transaction details

6. NFT Service (Port 8084)

Manages NFT lifecycle: minting, listing, buying, and trading. NFT metadata is stored in MongoDB. Trade events are published to Kafka. The service supports NFT statuses: MINTED, FOR_SALE, SOLD, BURNED.

■ NFT Lifecycle States

- MINTED — NFT created by user with name, description, image URL, and initial price
- FOR_SALE — Owner lists NFT on marketplace for purchase
- SOLD — Another user buys the NFT; ownership transferred
- BURNED — Owner permanently destroys NFT (removed from circulation)

■ Key Endpoints

Method	Path	Description
POST	/api/nft/mint	Mint a new NFT (name, price, imageUrl)
GET	/api/nft/marketplace	List all NFTs for sale
POST	/api/nft/trade/{tokenId}	Buy an NFT from marketplace
GET	/api/nft/my	Get NFTs owned by authenticated user

7. NACHA Service (Port 8085)

Implements US ACH (Automated Clearing House) payments conforming to NACHA rules. Supports PPD (consumer), CCD (business), and WEB (internet) SEC codes. Payments are processed asynchronously with T+1 settlement simulation.

■ NACHA Payment Flow

- User submits ACH payment with originator and receiver bank details
- Service generates unique Payment ID (NACHA-YYYYMMDD-NNNNNN)
- Trace number (15 digits) and batch number generated per NACHA spec
- Payment saved to MySQL with PENDING status
- Kafka event published to "nacha-payments" topic
- Async processor simulates ODFI processing (2s delay in demo)
- Status updated to SUBMITTED (95%) or FAILED with return code (5%)
- Real ACH settles T+1 (next business day)

■ Supported SEC Codes

Code	Full Name	Use Case
PPD	Prearranged Payment and Deposit	Consumer payroll, bill pay, direct deposit
CCD	Corporate Credit or Debit	B2B payments, vendor payments
WEB	Internet-Initiated Entry	Online bill payments, e-commerce

